7_tree.cpp

```
#include <iostream>
#include <queue>
using namespace std;

// Node structure
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int value) {
        data = value;
        left = NULL;
        right = NULL;
    }
};

// Insert into Binary Search Tree
Node* insert(Node* root, int value) {

    if (root == NULL)
        return new Node(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else
        root->right = insert(root->right, value);

    return root;
}

// Breadth-First Traversal (Level Order)
void levelOrder(Node* root) {

    if (root == NULL)
        return;

    queue<Node*> q;
    q.push(root);
```

```
    while (!q.empty()) {

        Node* current = q.front();
        q.pop();

        cout << current->data << " ";

        if (current->left != NULL)
            q.push(current->left);

        if (current->right != NULL)
            q.push(current->right);
    }
}

// Depth-First Traversal (Pre-order)
void preOrder(Node* root) {

    if (root == NULL)
        return;

    cout << root->data << " ";

    preOrder(root->left);
    preOrder(root->right);
}

int main() {

    Node* root = NULL;

    // Insert at least 7 nodes
    root = insert(root, 50);
    root = insert(root, 30);
    root = insert(root, 70);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 60);
    root = insert(root, 80);

    cout << "Breadth-First Traversal (Level Order): ";
    levelOrder(root);

    cout << endl;
```

```
    cout << "Depth-First Traversal (Pre-Order): ";
    preOrder(root);

    cout << endl;

    return 0;
}

/*
Time Complexity

Insertion: O(log n) average, O(n) worst case
Breadth-First Traversal: O(n)
Depth-First Traversal: O(n)
*/
```

Sample Output

Breadth-First Traversal (Level Order): 50 30 70 20 40 60 80

Depth-First Traversal (Pre-Order): 50 30 20 40 70 60 80

Tree Structure

The inserted values create this binary tree:

```
        50
       /    \
     30      70
    /  \    /  \
  20   40  60   80
```

Explanation

1. Node

struct Node

Each node stores:

- `data` → the value.
- `left` → pointer to the left child.
- `right` → pointer to the right child.

---

2. Insertion

insert(root, value);

- If the value is smaller than the current node, it goes to the **left**.
- If it is larger, it goes to the **right**.

This builds the Binary Search Tree.

---

3. Breadth-First Traversal (Level Order)

Visits nodes **level by level**.

Order for this tree:

50
30 70y
20 40 60 80

Output:

50 30 70 20 40 60 80

---

4. Depth-First Traversal (Pre-order)

Visits nodes in this order:

1. Root
2. Left subtree
3. Right subtree

Output:

50 30 20 40 70 60 80

Time Complexity

Operation	Complexity
Insertion	**O(log n)** average, **O(n)** worst case
Breadth-First Traversal	**O(n)**
Depth-First Traversal	**O(n)**